



Veille Sécurité — Projet Trouve Ton Artisan

1. Introduction

Dans le cadre du développement du site *Trouve Ton Artisan*, j'ai réalisé une veille sur les principales vulnérabilités Web afin de sécuriser l'API, le frontend et les données des utilisateurs. Cette veille m'a permis de comprendre les attaques les plus courantes, leurs impacts, et les bonnes pratiques pour les éviter.

2. SQL Injection (Injection SQL)

Définition L'injection SQL consiste à insérer du code SQL malveillant dans un champ de formulaire ou une URL afin de manipuler la base de données : suppression de tables, vol de données, contournement d'authentification...

Exemple d'attaque Un attaquant pourrait tenter : ' OR 1=1 -- pour contourner une connexion ou récupérer toutes les données.

Comment je l'ai évitée dans mon projet

- Utilisation de **Sequelize**, qui génère des requêtes préparées.
- Aucune requête SQL brute dans mon code.
- Validation des données côté backend avant insertion.
- Aucun champ utilisateur n'est directement injecté dans une requête.

Impact → Le risque d'injection SQL est **quasi nul** grâce à l'ORM et à la validation.

3. XSS (Cross-Site Scripting)

Définition Une attaque XSS consiste à injecter du JavaScript malveillant dans une page web afin de voler des cookies, rediriger l'utilisateur ou manipuler l'interface.

Exemple d'attaque Un utilisateur malveillant pourrait envoyer :
<script>alert("hack")</script>

Comment je l'ai évitée dans mon projet

- React échappe automatiquement le HTML, empêchant l'exécution de scripts.
- Aucun `dangerouslySetInnerHTML` utilisé.
- Validation des champs du formulaire côté backend.
- Les messages envoyés par les utilisateurs ne sont jamais affichés tels quels dans l'admin sans contrôle.

Impact → Le risque XSS est **fortement réduit** grâce à React et à la validation.

4. CSRF (Cross-Site Request Forgery)

Définition Une attaque CSRF force un utilisateur connecté à exécuter une action à son insu (ex : suppression de compte).

Pourquoi mon projet n'est pas vulnérable

- Le site n'utilise **pas de cookies de session**, mais un **JWT** envoyé dans les headers.
- Sans cookie automatique, un site tiers ne peut pas envoyer une requête authentifiée.
- Les routes sensibles (admin) sont protégées par un middleware qui vérifie le token.

Impact → Le risque CSRF est **quasi inexistant** grâce à l'utilisation du JWT.

5. Bonnes pratiques de sécurité mises en place

◆ Hash des mots de passe

- Utilisation de **bcrypt** pour hasher les mots de passe admin.
- Impossible de retrouver le mot de passe original même en cas de fuite.

◆ JWT sécurisé

- Signature avec une clé secrète.
- Expiration du token.
- Vérification systématique via un middleware.

◆ Validation des données

- Vérification des champs envoyés par les utilisateurs.
- Empêche les injections et les données invalides.

◆ CORS

- Autorise uniquement le frontend du projet à appeler l'API.
- Empêche les requêtes non autorisées.

◆ ORM Sequelize

- Requêtes préparées → protection SQL injection.
- Gestion automatique des paramètres.

◆ Routes admin protégées

- Impossible d'accéder au CRUD sans token valide.

- Sécurisation des actions sensibles.

6. Ce que j'ai appris grâce à cette veille

Cette veille m'a permis de comprendre :

- Que la majorité des attaques viennent de **données non contrôlées**.
- Que les ORM comme Sequelize protègent énormément contre les injections.
- Que React protège naturellement contre le XSS, mais qu'il faut rester vigilant.
- Que le JWT est une solution moderne et efficace pour éviter les attaques CSRF.
- Que la sécurité n'est pas un "bonus", mais une **obligation** dans un projet professionnel.
- Que chaque point du backend doit être pensé pour éviter les abus.

Cette veille m'a permis d'améliorer la qualité et la robustesse de mon projet, et de mieux comprendre les enjeux de la cybersécurité dans le développement web.



Documentation Sécurité — Projet Trouve Ton Artisan

1. Introduction

La sécurité est un élément essentiel dans le développement d'une application web. Dans *Trouve Ton Artisan*, plusieurs mesures ont été mises en place pour protéger :

- les données des utilisateurs,
- l'accès à l'administration,
- la base de données,
- les formulaires,
- les routes API.

Cette documentation détaille **chaque mécanisme de sécurité, comment il est implémenté, et pourquoi il est important.**

2. Hash des mots de passe (bcrypt)

◆ Mise en œuvre

Les mots de passe des administrateurs sont **hashés avec bcrypt** avant d'être enregistrés dans la base de données :

- utilisation de `bcrypt.hash()`
- salage automatique
- comparaison via `bcrypt.compare()` lors de la connexion

◆ Intérêt

- Un mot de passe hashé **ne peut pas être récupéré** même en cas de fuite.
- Empêche l'accès non autorisé à l'administration.
- Conforme aux bonnes pratiques OWASP.

3. Authentification sécurisée avec JWT

◆ Mise en œuvre

Le backend génère un **JSON Web Token** lors de la connexion admin :

- signé avec une clé secrète
- contient l'ID de l'admin

- expiration configurée
- envoyé dans les headers (Authorization: Bearer token)
- vérifié par un middleware sur toutes les routes protégées

◆ Intérêt

- Le JWT évite l'utilisation de cookies → réduit le risque CSRF.
- Impossible d'accéder au CRUD sans token valide.
- Permet une authentification stateless, simple et sécurisée.

4. Middleware d'authentification

◆ Mise en œuvre

Un middleware vérifie :

- la présence du token
- sa validité
- son expiration
- l'intégrité de la signature

Si le token est invalide → accès refusé.

◆ Intérêt

- Empêche l'accès aux routes sensibles (CRUD artisans, catégories, spécialités, messages).
- Centralise la logique de sécurité.
- Protège l'administration contre les intrusions.

5. Validation des données

◆ Mise en œuvre

Chaque formulaire (contact, admin, CRUD) est validé :

- champs obligatoires
- formats email / téléphone
- longueur minimale
- nettoyage des chaînes
- vérification côté backend avant insertion

♦ Intérêt

- Empêche les injections SQL.
- Empêche les attaques XSS via les champs texte.
- Garantit l'intégrité des données en base.

6. Protection contre les injections SQL (ORM Sequelize)

♦ Mise en œuvre

Le projet utilise **Sequelize**, un ORM qui :

- génère des requêtes préparées
- échappe automatiquement les valeurs
- empêche l'injection de code SQL
- interdit les concaténations dangereuses

♦ Intérêt

- Réduit drastiquement le risque d'injection SQL.
- Sécurise toutes les interactions avec MySQL.
- Conforme aux recommandations OWASP.

7. Protection contre les attaques XSS

♦ Mise en œuvre

- React échappe automatiquement le HTML.
- Aucun dangerouslySetInnerHTML utilisé.
- Les messages utilisateurs ne sont jamais affichés sans contrôle.
- Validation backend des champs texte.

♦ Intérêt

- Empêche l'exécution de scripts malveillants.
- Protège les utilisateurs et l'admin.
- Réduit les risques de vol de données ou de redirection forcée.

8. Protection contre les attaques CSRF

♦ Mise en œuvre

Le site utilise **JWT dans les headers**, pas de cookies.

♦ Intérêt

- Un site tiers ne peut pas envoyer une requête authentifiée.
- Le risque CSRF est quasiment nul.
- Conforme aux bonnes pratiques modernes.

9. CORS (Cross-Origin Resource Sharing)

♦ Mise en œuvre

Le backend autorise uniquement :

- le domaine du frontend
- les méthodes nécessaires
- les headers nécessaires (dont Authorization)

♦ Intérêt

- Empêche les requêtes provenant de domaines non autorisés.
- Protège l'API contre les appels externes malveillants.

10. Gestion des erreurs et retours sécurisés

♦ Mise en œuvre

- Messages d'erreur génériques côté client.
- Logs détaillés uniquement côté serveur.
- Pas de fuite d'informations sensibles (stack, SQL, chemins internes).

♦ Intérêt

- Empêche un attaquant d'obtenir des informations techniques.
- Améliore la robustesse de l'application.

11. Sécurisation des routes API

♦ Mise en œuvre

- Routes publiques : artisans, catégories, spécialités
- Routes protégées : CRUD + messages
- Middleware d'authentification obligatoire
- Vérification des paramètres d'URL

◆ Intérêt

- Empêche la modification des données sans autorisation.
- Sécurise l'administration et la base de données.

12. Conclusion

La sécurité a été intégrée à chaque étape du projet :

- Authentification robuste
- Protection contre les injections
- Validation des données
- Sécurisation des routes
- Gestion des erreurs
- Sécurité frontend (React)
- Sécurité backend (JWT, bcrypt, ORM)

Le projet respecte les bonnes pratiques essentielles recommandées par **OWASP** et garantit un niveau de sécurité adapté à une application professionnelle.